results from the Agents4Science conference and include the author evaluation, which helps clarify what is required to further improve the quality of Jr. AI Scientist systems.

Finally, we perform an in-depth report of the risks encountered during the development of our system. Although few existing studies have provided a comprehensive discussion of these issues, we believe that accurately documenting such risks is essential to avoid overestimating current AI Scientists' capabilities and to build a clear understanding of their remaining challenges. Our risk report highlights several critical issues, including the potential for review-score hacking and difficulties in ensuring proper citation, interpreting results, and detecting fabricated descriptions. We believe that these findings provide valuable guidance on the potential risks that exist both in the current AI Scientist research and as this field continues to grow. Through this comprehensive report, we aim to foster a deeper understanding of current AI Scientist systems and contribute to their safe and trustworthy development.

Our contributions are summarized as follows:

- **Development of a New AI Scientist**: We developed Jr. AI Scientist, a new system that starts from a baseline paper and its associated codebase, and is capable of handling complex, multi-file implementations, overcoming a major limitation of previous AI Scientist systems.
- **Revealing Strengths and Limitations of Jr. AI Scientist**: We conducted extensive evaluations using open-source AI reviewers, Agents4Science, and author evaluation. The results demonstrate that Jr. AI Scientist generates higher-quality research papers than existing AI Scientists, while also revealing key challenges for future improvement.
- **Thorough Risk Report**: We report the observed risks throughout the project. We believe these reports offer insights into the areas that still require human expertise and the risks that may emerge as these systems evolve.

2 Related Work

Automated Scientific Discovery. Recent progress has significantly reshaped the role of AI in automating end-to-end scientific research. AI Scientist-v1 (Lu et al., 2024) was an early milestone, showcasing how advanced language models can autonomously generate research ideas, run experiments, and draft scientific papers. This work was followed by a series of subsequent studies in machine learning fields (Intology, 2025; Tang et al., 2025) and diverse scientific disciplines (Villaescusa-Navarro et al., 2025; Mitchener et al., 2025) that further advanced this line of research. However, these approaches often suffer from an overly ambitious problem setting that aims to achieve fully automated science and tend to lack clearly defined scientific objectives for AI Scientists. Without specific goals, such systems often produce undirected discoveries that lack genuine scientific value. To address this issue, our Jr. AI Scientist builds on existing baselines and conducts research within a well-defined research workflow, aiming to generate higher-quality scientific papers. As concurrent work, DeepScientist (Weng et al., 2025b) also adopts a baseline-based approach. DeepScientist (Weng et al., 2025b) focuses mainly on experimental performance, formalizing discovery as a Bayesian Optimization problem. However, its overall framework design and workflow integration are outlined at a high level, with limited discussion of implementation details. In contrast, our Jr. AI Scientist explicitly articulates each stage of the research process and further aims to contribute to the community by comprehensively reporting the failures and risks encountered throughout scientific exploration.

AI-Assisted Scientific Research. Research specialized for each element of the research process has also been actively explored (Chen et al., 2025). For the idea generation phase, Si et al. (2025b) investigates the novelty of the LLM-generated ideas, and Si et al. (2025a) investigates the ideation–execution gap. For the survey phase, OpenScholar (Asai et al., 2026) have been developed to support literature review. For the experimental phase, AlphaEvolve (Novikov et al., 2025) leverages large-scale trial-and-error strategies to enhance the performance. For the writing and review phase, CycleResearcher (Weng et al., 2025a) provides a learning framework specialized for scientific writing, while DeepReviewer (Zhu et al., 2025a) focuses on the review process. Recent studies provide a comprehensive overview of automated review, outlining key challenges, proposing a practical review pipeline for real-world implementation, and constructing a large-scale

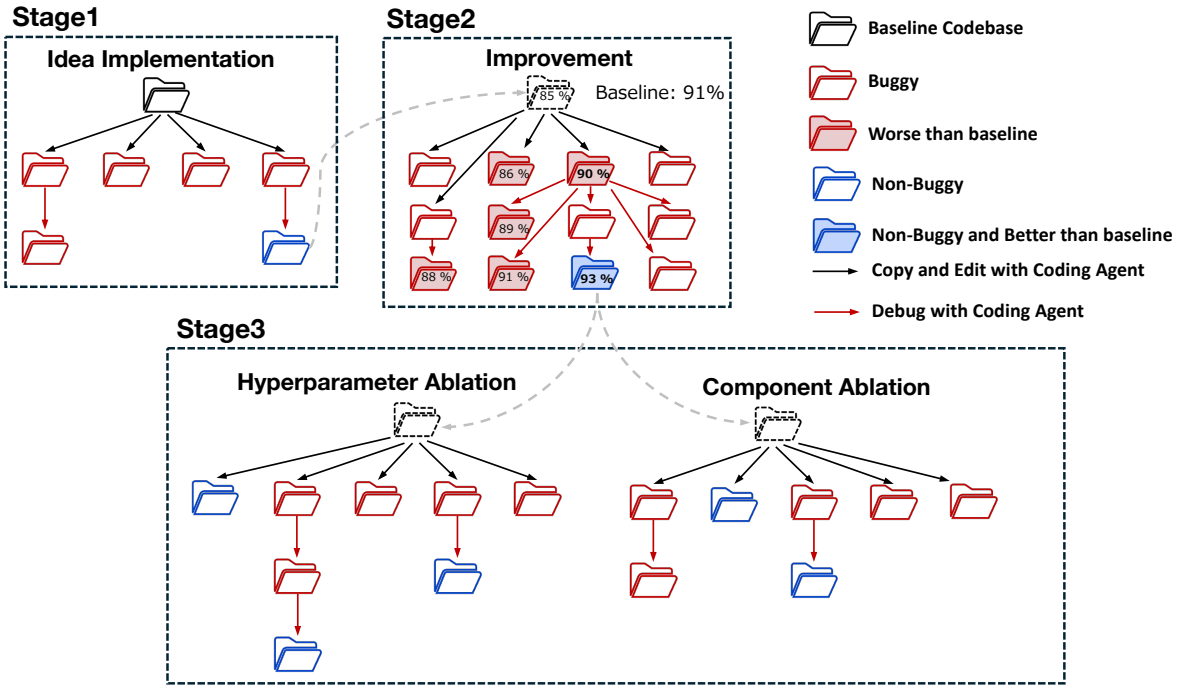

Figure 2: **Jr. AI Scientist Workflow for the Experiment Phase.** The workflow consists of three stages. Through bug management and performance tracking, our system passes the most promising experimental nodes to the next stage.

dataset to support automated review research (Zhuang et al., 2025; Lin et al., 2023a;b). Beyond these, rather than pursuing full automation like AI Scientists, AI Co-Scientist (Gottweis et al., 2025) emphasizes collaboration between humans and AI. In this work, instead of focusing on individual parts of the research process, we investigate the entire end-to-end research cycle, aiming to rigorously evaluate both the performance and the limitations.

Failures and Risk Analysis for AI Scientist Systems. There are very few studies that thoroughly analyze or report the risks and failure cases of AI Scientist systems. Although Tang et al. (2024) summarizes the risks that AI Scientists may pose, it focuses on hypothesis-based potential risks, rather than empirically observed risks or failures. While Beel et al. (2025) provides an in-depth analysis of failure cases in AI Scientist-v1 (Lu et al., 2024), these findings are based on the early AI Scientist, and the analysis was not conducted from the developer's perspective. While Luo et al. (2025) examines four failure modes (benchmark selection, data leakage, metric misuse, and post-hoc bias), their analysis is limited to experimental diagnostics and early AI Scientists without modern coding. Thus, their analysis remains somewhat limited in scope.

Therefore, we will provide a more comprehensive report on the various risks identified during the development of our state-of-the-art AI Scientist, in order to deepen the community's understanding of AI Scientists.

3 Jr. AI Scientist

In this section, we describe the mechanisms behind the three components of the Jr. AI Scientist: idea generation, experimentation, and writing. First, in Section 3.1, we describe the necessary preparation. Next, in Section 3.2, we explain the methods for idea generation. Next, in Section 3.3, we discuss how agents can execute and manage experiments. Finally, in Section 3.4, we explain the writing process.

3.1 Preparation: Baseline Paper Selection

The preparation stage involves selecting a baseline paper, obtaining its LaTeX source files and PDF, and the baseline code. This setup is realistic because many recent publications are released on arXiv with LaTeX

sources, and their implementation code is shared on GitHub. While there might be some augment that AI agents should automatically select a baseline and reproduce the code, current reproducibility rates from papers are still limited (Siegel et al., 2025; Xiang et al., 2025; Starace et al., 2025), making complete automation premature. Since our goal is to emulate how a human scientist engages in early-stage research under the guidance of a mentor, we explicitly include this preparatory stage.

When constructing the baseline code, we followed AI Scientist v1 (Lu et al., 2024) and made only minor modifications to the existing implementation so that the experiments could be executed via `baseline.py` and the results could be visualized via `plot.py`. Defining such an experimental entry point facilitated easier management and reproducibility of the execution process in the experimental section.

3.2 Idea Generation Phase

We provide a baseline paper as text to an LLM (e.g. o4-mini (OpenAI, 2025b)) and prompt it to output the limitations of the work. Based on both the baseline paper and the limitations, the LLM is then guided to propose potential research ideas. Following AI Scientist v2 (Yamada et al., 2025), the system evaluates the originality of proposed ideas through literature review tools such as Semantic Scholar, which review papers citing the baseline work and papers with similar concepts. If conceptually similar ideas are identified, they are refined; otherwise, they are clearly distinguished from prior work. These steps define the preliminary research idea.

3.3 Experiment Phase

The Experiment Phase mainly involves implementing and iterating on the implementations through experiments. It is divided into three stages: Stage 1: Idea Implemention, Stage 2: Iterative Improvement, and Stage 3: Ablation Study. The workflow of each stage is shown in Figure 2. We first describe the general procedure for using the coding agent, followed by an explanation of the implementation at each stage.

3.3.1 Preliminary: Coding Agent Usage

A powerful coding agent (e.g. Claude Code (Anthropic, 2025)) is employed to translate research ideas into concrete implementations. We provide the coding agent with a working directory that contains the baseline implementations (prepared at Section 3.1) and give it detailed instructions through input prompts. The agent is informed of how to use the main scripts—`baseline.py`, which serves as the experimental entry point, and `plot.py`, which visualizes the experimental results. We use `claude-sonnet-4-20250514` within Claude Code (version 1.0.24).

The coding agent is allowed to read and write any files within the working directory. For efficient directory exploration, it is permitted to use commands such as `ls` and `grep`, while commands that may cause side effects (e.g. `python` or other execution commands) are not allowed. After the agent generates a runnable file (e.g. `proposed_method.py` in Stage1), our system mechanically executes the specified command. The coding agent is generally given up to 30 turns to complete each assigned task.

3.3.2 Stage1: Idea Implemention

The system manages four experimental nodes running in parallel, each responsible for implementing and testing a proposed idea independently. Within each node, the coding agent receives the baseline code and a research idea, and writes a directly executable script named `proposed_method.py`. Once the coding agent finishes writing the implementation, the system sequentially runs `proposed_method.py` and `plot.py`. If a result file is successfully generated, the codebase is marked as Non-Buggy; otherwise, it is labeled as Buggy If a visualization image is also produced, it is further marked as Non-Plot-Buggy; otherwise, as Plot-Buggy. Each iteration of this process, coding and execution, is counted as one trial and is repeated until a bug-free implementation is obtained.

As shown in Figure 2, if any node completes successfully without encountering bugs, its codebase is carried forward to Stage 2. During execution, four experimental nodes are selected. If all currently running nodes fail,